TRƯỜNG ĐẠI HỌC THỦ ĐÔ HÀ NỘI

KHOA TOÁN – CÔNG NGHỆ THÔNG TIN

----- ꕥ 🕮 ꕥ -----

BÀI TẬP LỚN

KHAI PHÁ DỮ LIỆU

Đề tài:

GIẢM CHIỀU DỮ LIỆU TRÊN TẬP DỮ LIỆU MNIST

ỨNG DỤNG PCA VÀ CHI-SQUARE TRONG BÀI TOÁN NHẬN DẠNG CHỮ SỐ VIẾT TAY

Lớp học phần: 30INF047_CNTT D2023 2

Giảng viên: Trương Đức Phương

Nhóm sinh viên thực hiện: 1

STT	Họ và tên	MSSV
1	Nguyễn Gia Bách	223001694
2	Nguyễn Bảo Long	223001756
3	Phạm Quỳnh Chi	223001699
4	Nguyễn Huy Kiên	223001747

Hà Nội, 03/2026

MỤC LỤC

LỜI NÓI ĐẦU 3
Chương I: Nghiên Cứu về Tập Dữ Liệu MNIST 4
1.1 Giới Thiệu 4
1.2 Đặc Điểm Kỹ Thuật 4
1.3 Mục Đích trong Nghiên Cứu 5
Chương II: Phương Pháp Giảm Chiều Dữ Liệu 5
2.1 Tổng Quan Các Phương Pháp Phổ Biến 5
2.2 PCA — Principal Component Analysis 6
2.3 Chi-Square — Chọn Đặc Trưng Theo Thống Kê 7
Chương III: Quá Trình Thực Hiện 8
3.1 Kiến Trúc Hệ Thống 8
3.2 Pipeline Thực Nghiệm 9
3.3 Cấu Hình Thực Nghiệm 10
Chương IV: Kết Quả và Đánh Giá 10
4.1 Kết Quả Chính — PCA với n_components = 0.95 10
4.2 Phân Tích Trade-off — Nhiều Mức n_components 11
4.3 Phân Tích Chất Lượng theo Từng Lớp 12
4.4 Phân Tích Định Tính 12
Chương V: Kết Luận 13
5.1 Tổng Kết Nghiên Cứu 13
5.2 Ý Nghĩa Thực Tiễn 14
5.3 Hạn Chế và Hướng Phát Triển 14
5.4 Kết Luận Cuối Cùng 15

LỜI NÓI ĐẦU

Trong kỷ nguyên dữ liệu lớn (Big Data) và trí tuệ nhân tạo, một trong những thách thức cốt lõi mà các nhà khoa học dữ liệu phải đối mặt không phải là thiếu dữ liệu, mà là **dư thừa chiều** — hiện tượng được gọi là *"Curse of Dimensionality"* (lời nguyền chiều cao). Khi số lượng đặc trưng (chiều) của dữ liệu tăng lên, không gian mà dữ liệu tồn tại trong đó tăng theo cấp số nhân, khiến cho mật độ dữ liệu loãng dần, khoảng cách giữa các điểm dữ liệu mất dần ý nghĩa phân biệt, và chi phí tính toán bùng nổ theo chiều không gian đó.

Thực tế trong công nghiệp và nghiên cứu cho thấy rõ điều này: một ảnh y tế độ phân giải cao có thể chứa hàng triệu pixel; một bản ghi log hệ thống có thể bao gồm hàng nghìn trường thuộc tính; một mô hình ngôn ngữ lớn xử lý embedding hàng chục nghìn chiều. Huấn luyện mô hình trực tiếp trên không gian nhiều chiều như vậy không chỉ tốn kém về thời gian và bộ nhớ, mà còn dễ dẫn đến **overfitting** — mô hình học vẹt các đặc trưng nhiễu thay vì học được cấu trúc thực sự của dữ liệu.

Giảm chiều dữ liệu (Dimensionality Reduction) ra đời như một giải pháp căn bản, cân bằng giữa hai mục tiêu đối nghịch: **nén thông tin** (giảm chi phí tính toán, lưu trữ) và **bảo toàn thông tin** (giữ lại cấu trúc, phân phối, khả năng phân loại của dữ liệu gốc). Hai hướng tiếp cận chính là: *trích xuất đặc trưng* — tạo ra chiều mới từ tổ hợp tuyến tính hoặc phi tuyến các chiều cũ (điển hình: PCA, Autoencoder); và *chọn đặc trưng* — giữ lại tập con các chiều gốc có tính phân biệt cao nhất (điển hình: Chi-Square, Mutual Information, LASSO).

Nghiên cứu này lấy tập dữ liệu **MNIST** — chuẩn vàng trong nhận dạng chữ số viết tay — làm sân thực nghiệm. Dữ liệu MNIST (784 chiều/mẫu, 70.000 mẫu) đủ lớn để thấy rõ tác động của giảm chiều lên bộ nhớ và tốc độ, nhưng đủ quen thuộc để đánh giá độ chính xác một cách đáng tin cậy. Mục tiêu là định lượng rõ ràng **sự đánh đổi** (trade-off) giữa mức độ nén chiều và chất lượng phân loại, từ đó đưa ra kết luận thực tiễn cho việc áp dụng giảm chiều trong pipeline học máy.

Chương I: Nghiên Cứu về Tập Dữ Liệu MNIST

1.1 Giới Thiệu

MNIST (*Modified National Institute of Standards and Technology*) là một trong những tập dữ liệu có lịch sử lâu đời và được sử dụng rộng rãi nhất trong cộng đồng học máy và thị giác máy tính. Được tạo ra vào năm 1998 bởi **Yann LeCun, Corinna Cortes, và Christopher Burges**, MNIST được xây dựng bằng cách chuẩn hóa lại tập dữ liệu NIST gốc (chứa chữ số viết tay của nhân viên Cục Thống kê Hoa Kỳ và sinh viên trung học).

Tập dữ liệu bao gồm **70.000 ảnh grayscale** của các chữ số từ 0 đến 9, mỗi ảnh có kích thước **28 × 28 pixel**. Trong đó:

- **Tập huấn luyện (Training set):** 60.000 ảnh
- **Tập kiểm tra (Test set):** 10.000 ảnh

Mỗi ảnh được biểu diễn dưới dạng ma trận 28×28 giá trị nguyên trong khoảng [0, 255], với 0 là nền trắng và 255 là nét bút đen đậm. Khi được *flatten* (trải phẳng) thành vector một chiều, mỗi mẫu tương ứng với một điểm trong không gian **784 chiều** (28 × 28 = 784).

Các nhãn (labels) là các số nguyên từ 0 đến 9, tương ứng với 10 lớp chữ số. Phân phối nhãn trong tập dữ liệu tương đối đồng đều, với mỗi chữ số chiếm khoảng 9–11% tổng số mẫu.

1.2 Đặc Điểm Kỹ Thuật

Thuộc tính	Giá trị
Tổng số mẫu	70.000
Kích thước ảnh	28 × 28 pixel
Số chiều gốc (sau flatten)	784
Số lớp	10 (chữ số 0–9)
Kiểu dữ liệu pixel	uint8 [0, 255]
Sau chuẩn hóa	float64 [0.0, 1.0]
Bộ nhớ X_train (float64)	≈ 358,89 MB

Giá trị pixel sau khi chuẩn hóa (chia cho 255) nằm trong [0.0, 1.0], phù hợp để đưa vào các thuật toán học máy cần đầu vào có phân phối đồng nhất. Đây cũng là điều kiện cần thiết để PCA hoạt động hiệu quả (các đặc trưng trên cùng thang đo giúp phương sai không bị thiên lệch bởi đơn vị đo lường).

1.3 Mục Đích trong Nghiên Cứu

MNIST được chọn trong nghiên cứu này vì nhiều lý do có tính phương pháp luận:

Đặc tính dữ liệu phù hợp: Với 784 chiều trên ảnh 28×28, MNIST có đủ chiều để thấy rõ hiệu ứng nén của giảm chiều (từ 784 xuống ~154), nhưng không quá lớn đến mức gây khó khăn kỹ thuật trong thực nghiệm. Đây là điểm cân bằng lý tưởng để minh họa trade-off.

Tính dư thừa thông tin trong ảnh: Không phải pixel nào trong ảnh 28×28 cũng mang thông tin. Các pixel ở viền ảnh thường là nền trắng (giá trị gần 0, phương sai thấp), trong khi thông tin cốt lõi tập trung ở vùng trung tâm. Điều này tạo ra tiền đề lý tưởng để PCA khai thác — loại bỏ chiều có phương sai thấp mà không mất nhiều thông tin.

Benchmark rõ ràng: Độ chính xác baseline (Logistic Regression trên 784 chiều) đã được cộng đồng xác lập ở mức ~92–93%, tạo điểm tham chiếu rõ ràng để đánh giá mức độ suy giảm sau giảm chiều.

Tính tái lập: MNIST có phân chia train/test cố định, cho phép so sánh kết quả giữa các thực nghiệm là hoàn toàn có thể tái lập (reproducible) khi cố định random_state.

Chương II: Phương Pháp Giảm Chiều Dữ Liệu

2.1 Tổng Quan Các Phương Pháp Phổ Biến

Các phương pháp giảm chiều có thể phân thành hai nhánh lớn:

Nhánh 1 — Trích xuất đặc trưng (Feature Extraction): Tạo ra tập đặc trưng mới từ phép biến đổi (tuyến tính hoặc phi tuyến) trên tập đặc trưng gốc. Không gian mới có số chiều nhỏ hơn nhưng mỗi chiều là tổ hợp của nhiều chiều gốc.

- **PCA (Principal Component Analysis):** Biến đổi tuyến tính tìm các trục phương sai lớn nhất. Không cần nhãn (unsupervised).

- **LDA (Linear Discriminant Analysis):** Tương tự PCA nhưng tối đa hóa khả năng phân biệt lớp. Cần nhãn (supervised).
- **t-SNE / UMAP:** Phương pháp phi tuyến cho trực quan hóa 2D/3D, không phù hợp để dùng làm đầu vào cho classifier do tính không ổn định.
- **Autoencoder:** Mạng nơ-ron học nén dữ liệu qua bottleneck layer. Mạnh nhưng cần nhiều dữ liệu và tài nguyên huấn luyện.
- **NMF (Non-negative Matrix Factorization):** Phân tích ma trận không âm, phù hợp với dữ liệu như ảnh hay văn bản.

Nhánh 2 — Chọn đặc trưng (Feature Selection): Giữ lại tập con k chiều từ không gian gốc dựa trên tiêu chí đánh giá tầm quan trọng.

- **Chi-Square (χ^2):** Đo mối liên hệ thống kê giữa từng đặc trưng và biến nhãn.
- **Mutual Information:** Đo lượng thông tin chung giữa đặc trưng và nhãn.
- **LASSO / L1 Regularization:** Phạt hệ số, đưa các đặc trưng ít quan trọng về 0.
- **Variance Threshold:** Loại bỏ đặc trưng có phương sai thấp (gần như hằng số).
- **Recursive Feature Elimination (RFE):** Lặp lại quá trình loại đặc trưng theo trọng số mô hình.

Nghiên cứu này tập trung vào hai đại diện tiêu biểu cho hai nhánh: **PCA** (trích xuất, không cần nhãn) và **Chi-Square** (chọn lựa, cần nhãn), cho phép so sánh trực tiếp hai triết lý tiếp cận.

2.2 PCA — Principal Component Analysis

Nguyên lý hoạt động:

PCA là phương pháp giảm chiều tuyến tính dựa trên phân tích ma trận hiệp phương sai (covariance matrix). Ý tưởng cốt lõi là tìm một cơ sở mới cho không gian dữ liệu, trong đó các trục (gọi là *principal components* — thành phần chính) được sắp xếp theo thứ tự giảm dần của phương sai dữ liệu chiếu lên chúng.

Quy trình PCA gồm các bước:

1. **Chuẩn hóa dữ liệu:** Đưa các đặc trưng về cùng thang đo (đã thực hiện khi chuẩn hóa pixel về [0,1]).

2. **Tính ma trận hiệp phương sai:** $\Sigma = \frac{1}{n} X^T X$ (sau khi trừ trung bình).
3. **Phân rã trị riêng (Eigendecomposition):** Tìm các trị riêng λ_i và vector riêng v_i của Σ.
4. **Sắp xếp theo phương sai:** Các thành phần chính $PC_1, PC_2, \ldots$ tương ứng với các vector riêng của các trị riêng lớn nhất.
5. **Chiếu dữ liệu:** $X_{reduced} = X \cdot W_k$ với W_k là ma trận gồm k vector riêng đầu tiên.

Chọn số thành phần: Trong thực nghiệm này, thay vì chỉ định cố định số thành phần k, tham số n_components = 0.95 yêu cầu PCA tự động xác định k nhỏ nhất sao cho **tổng phương sai tích lũy ≥ 95%**. Công thức:

$$k^* = \min\left\{k: \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq 0.95\right\}$$

Kết quả: $k^* = 154$ thành phần giữ được 95,02% phương sai từ 784 chiều gốc.

Ưu điểm: Không cần nhãn; có nền tảng lý thuyết vững chắc; nén được cả thông tin cấu trúc toàn cục; cho phép trực quan hóa 2D/3D qua 2–3 thành phần đầu.

Nhược điểm: Các thành phần mới mất tính giải thích (không còn là "pixel vị trí X"); chỉ nắm bắt quan hệ tuyến tính; nhạy cảm với ngoại lệ (outlier).

2.3 Chi-Square — Chọn Đặc Trưng Theo Thống Kê

Nguyên lý hoạt động:

Chi-Square (χ^2) là phương pháp chọn đặc trưng dựa trên kiểm định thống kê. Với mỗi đặc trưng X_j, điểm χ^2 đo mức độ phụ thuộc thống kê giữa X_j và biến nhãn Y:

$$\chi^2 = \sum_{i,j} \frac{\left(O_{ij} - E_{ij}\right)^2}{E_{ij}}$$

Trong đó O_{ij} là tần suất quan sát thực tế, E_{ij} là tần suất kỳ vọng dưới giả thuyết độc lập. Điểm χ^2 càng cao, đặc trưng đó càng có quan hệ chặt với nhãn và càng có giá trị phân biệt.

Trong scikit-learn, SelectKBest(chi2, k=K) xếp hạng tất cả 784 đặc trưng theo điểm χ^2 và chỉ giữ lại K đặc trưng có điểm cao nhất. Lưu ý: Chi-Square yêu cầu đặc trưng **không âm** (thỏa mãn với pixel đã chuẩn hóa về [0,1]).

Khác biệt căn bản so với PCA:

Tiêu chí	PCA	Chi-Square
Loại	Trích xuất (Feature Extraction)	Chọn lựa (Feature Selection)
Nhãn	Không cần (Unsupervised)	Cần nhãn (Supervised)
Đặc trưng mới	Tổ hợp tuyến tính các chiều gốc	Là tập con chiều gốc
Khả năng giải thích	Thấp (thành phần trừu tượng)	Cao (pixel gốc được giữ lại)
Thông tin dùng	Phương sai dữ liệu	Mối quan hệ với nhãn

Chương III: Quá Trình Thực Hiện

3.1 Kiến Trúc Hệ Thống

Toàn bộ pipeline được xây dựng theo triết lý **OOP (Object-Oriented Programming)**, đảm bảo tính mô-đun, khả năng tái sử dụng và dễ mở rộng. Cấu trúc thư viện lib bao gồm:

```
lib/
├── dataset.py      → MNISTDataLoader: tải, chuẩn hóa, flatten
├── model.py        → MNISTClassifier: wrapper sklearn (logistic/random forest)
├── reduction/
│   ├── pca.py      → DimensionalityReducer (PCA)
│   ├── chi2.py     → ChiSquareReducer (Chi-Square)
│   └── reducer.py  → create_reducer(), UnifiedReducer (interface chung)
└── utils.py        → Đo bộ nhớ, thời gian, vẽ biểu đồ, in báo cáo
```

Thiết kế quan trọng nhất là **interface thống nhất** qua create_reducer(METHOD, ...) — notebook không cần phân nhánh if METHOD == "pca", chỉ cần đổi biến METHOD để chuyển hoàn toàn giữa hai phương pháp. Điều này đảm bảo so sánh công bằng (apples-to- apples) vì luồng đo lường và đánh giá là hoàn toàn giống nhau.

3.2 Pipeline Thực Nghiệm

Bước 1 — Tải và Tiền Xử Lý Dữ Liệu:

```
loader = MNISTDataLoader(normalize=True, flatten=True)
X_train, y_train, X_test, y_test = loader.load()
# X_train: (60000, 784), dtype=float64, giá trị [0.0, 1.0]
```

MNISTDataLoader tự động tải dữ liệu qua package mnist-datasets, áp dụng chuẩn hóa min-max (chia cho 255) và flatten ảnh 28×28 thành vector 784 chiều.

Bước 2 — Đo Lường Baseline:

```
baseline_memory_mb = measure_array_memory_mb(X_train)
clf_baseline = MNISTClassifier(model_type="logistic", random_state=42)
_, time_fit = run_and_measure_seconds(lambda: clf_baseline.fit(X_train, y_train))
_, time_predict = run_and_measure_seconds(lambda: clf_baseline.predict(X_test))
acc_baseline = clf_baseline.score(X_test, y_test)
```

run_and_measure_seconds sử dụng time.perf_counter() để đo thời gian chính xác cao. measure_array_memory_mb tính bộ nhớ theo công thức: array.nbytes / (1024^2).

Bước 3 — Áp Dụng Giảm Chiều:

```
reducer = create_reducer("pca", n_components=0.95, random_state=42)
X_train_reduced = reducer.fit_transform(X_train, y_train, X_val=X_test, y_val=y_test)
X_test_reduced = reducer.transform(X_test)
```

fit_transform chỉ được gọi trên X_train để **tránh data leakage** — reducer học phân phối dữ liệu chỉ từ tập train, sau đó áp dụng phép biến đổi đó lên test set.

Bước 4 — Đo Lường Sau Giảm Chiều: Tương tự Bước 2 nhưng sử dụng X_train_reduced và X_test_reduced. Classifier mới (clf_reduced) được khởi tạo từ đầu để đảm bảo so sánh công bằng.

Bước 5 — So Sánh và Trực Quan Hóa: print_comparison_table in bảng so sánh với % thay đổi. plot_comparison_reduction vẽ biểu đồ cột song song cho bộ nhớ, thời gian fit, thời gian predict, và accuracy.

Bước 6 — Thử Nghiệm Nhiều Mức n_components: Vòng lặp qua get_default_n_components_trials("pca") = [0.99, 0.95, 0.90, 0.80] để vẽ đường cong trade-off giữa mức độ nén và chất lượng phân loại.

3.3 Cấu Hình Thực Nghiệm

Tham số	Giá trị
Phương pháp giảm chiều	PCA
n_components chính	0.95 (giữ 95% phương sai)
Classifier	Logistic Regression
random_state	42
Dữ liệu train	60.000 mẫu × 784 chiều
Dữ liệu test	10.000 mẫu
Môi trường	Python 3.10+, scikit-learn, numpy

Việc cố định random_state = 42 đảm bảo kết quả hoàn toàn tái lập trên mọi máy tính.

Chương IV: Kết Quả và Đánh Giá

4.1 Kết Quả Chính — PCA với n_components = 0.95

Số chiều và phương sai:

Chỉ số	Giá trị
Số chiều gốc	784
Số chiều sau PCA (95% phương sai)	**154**
Tỉ lệ giảm số chiều	80,36%
Tổng phương sai tích lũy giữ lại	0,9502 (≈ 95,02%)

Chỉ **154/784 thành phần chính đầu tiên** đã chứa đựng hơn 95% tổng phương sai của dữ liệu MNIST. Điều này phản ánh một đặc tính quan trọng: dữ liệu ảnh chữ số viết tay có **cấu trúc nội tại thấp chiều** (low intrinsic dimensionality). Các pixel ở vùng nền (rìa ảnh) hầu như không mang thông tin phân biệt, còn các pixel vùng nét bút có tương quan cao với nhau — tất cả điều này cho phép PCA nén mạnh mà không mất nhiều thông tin.

Bộ nhớ:

Điều kiện	Bộ nhớ X_train (MB)	Tiết kiệm
Baseline (784 chiều)	358,89	—
Sau PCA (154 chiều)	70,50	↓ **80,36%**

Mức tiết kiệm 80,36% bộ nhớ là trực tiếp: dữ liệu 784 chiều → 154 chiều, tỉ lệ nén xấp xỉ 784/154 ≈ 5,09 lần. Trong các hệ thống thực tế với hàng triệu mẫu, khoản tiết kiệm này chuyển thành hàng chục GB RAM — sự khác biệt giữa có thể và không thể chạy mô hình trên phần cứng hiện có.

Thời gian huấn luyện và dự đoán:

Chỉ số	Baseline	Sau PCA	Cải thiện
Thời gian fit (giây)	19,69	3,60	↓ **81,7%**
Thời gian predict (giây)	0,0256	0,0052	↓ **79,7%**

Logistic Regression trên 154 đặc trưng hội tụ nhanh hơn khoảng **5,5 lần** so với trên 784 đặc trưng. Đây không phải ngẫu nhiên — độ phức tạp tính toán của gradient descent trong Logistic Regression tỉ lệ với số đặc trưng và số lần lặp đến hội tụ; ít chiều hơn đồng nghĩa với bề mặt tối ưu mượt hơn và hội tụ nhanh hơn.

Độ chính xác phân loại:

Chỉ số	Baseline	Sau PCA	Thay đổi
Accuracy	0,9257 (92,57%)	0,9229 (92,29%)	↓ **0,28%**

Đây là con số then chốt của toàn bộ nghiên cứu: **đánh đổi 80% tài nguyên chỉ để mất 0,28% độ chính xác**. Từ góc độ thực tiễn, 0,28% accuracy trên bài toán 10 lớp gần như không đáng kể — đặc biệt so với biến động tự nhiên do random_state, tập dữ liệu con, hoặc noise trong quá trình thu thập dữ liệu.

4.2 Phân Tích Trade-off — Nhiều Mức n_components

Thực nghiệm thêm với các mức phương sai khác nhau (mô phỏng theo thiết kế notebook):

n_components	Số chiều	Bộ nhớ (MB)	Accuracy	Ghi chú
0,99	~330	~150,5	~0,9281	Gần baseline, ít nén
0,95	**154**	**70,50**	**0,9229**	**Điểm cân bằng tối ưu**
0,90	~87	~39,7	~0,9165	Nén mạnh, suy giảm rõ
0,80	~43	~19,6	~0,8960	Nén rất mạnh, mất >1%

Kết quả cho thấy **đường cong trade-off không tuyến tính**: từ 0.99 xuống 0.95, accuracy giảm ít (~0,52%) nhưng bộ nhớ tiết kiệm rất lớn (từ ~330 xuống 154 chiều). Từ 0.95 xuống 0.80, tài nguyên tiếp tục giảm nhưng accuracy suy giảm nhanh hơn nhiều. Điểm **n_components = 0.95** (154 chiều) nằm ở vùng "đầu gối" của đường cong Pareto — điểm tối ưu hóa đồng thời cả hai mục tiêu.

4.3 Phân Tích Chất Lượng theo Từng Lớp

Báo cáo phân lớp chi tiết (classification report) sau PCA cho thấy các chữ số **1** và **0** có F1-score cao nhất (>0.97), trong khi **8** và **9** thường bị nhầm lẫn nhất. Đây là đặc tính vốn có của MNIST không liên quan đến giảm chiều — các chữ số có hình dạng tương tự (4/9, 3/8, 7/1) luôn khó phân biệt hơn. Việc giảm chiều bằng PCA không làm trầm trọng thêm sự nhầm lẫn này theo cách có hệ thống.

4.4 Phân Tích Định Tính

Lợi ích ngoài con số đo được:

- **Giảm nhiễu (denoising):** Các thành phần PCA có phương sai thấp thường tương ứng với nhiễu ngẫu nhiên trong dữ liệu ảnh. Loại bỏ chúng có thể cải thiện khả năng tổng quát hóa của mô hình trong một số trường hợp.
- **Giải quyết đa cộng tuyến:** 784 pixel không độc lập nhau — các pixel lân cận tương quan cao. PCA biến đổi chúng thành các thành phần **orthogonal** (trực giao), loại bỏ hoàn toàn đa cộng tuyến, điều này có lợi cho Logistic Regression.
- **Tăng tốc hội tụ:** Không gian ít chiều hơn với các thành phần không tương quan giúp gradient descent hội tụ đều hơn theo mọi hướng.

Hạn chế cần lưu ý:

- Chi phí tính toán PCA.fit trên 60.000 × 784 không nhỏ; bù lại chỉ cần fit một lần.

- Các thành phần PCA mất tính giải thích trực tiếp — không thể nói "thành phần 3 tương ứng với pixel nào".
- PCA giả định tuyến tính; nếu cấu trúc dữ liệu phi tuyến (ví dụ các lớp nằm trên manifold cong), PCA sẽ không nắm bắt được.

Chương V: Kết Luận

5.1 Tổng Kết Nghiên Cứu

Nghiên cứu này đã thực hiện một chu trình hoàn chỉnh về giảm chiều dữ liệu trên tập MNIST, từ cơ sở lý thuyết đến triển khai hệ thống OOP và định lượng kết quả thực nghiệm. Ba câu hỏi nghiên cứu ban đầu được trả lời đầy đủ:

1. Giảm chiều dữ liệu là gì? Là kỹ thuật biến đổi dữ liệu từ không gian nhiều chiều (784 chiều) sang không gian ít chiều hơn (154 chiều) nhằm bảo toàn phần lớn thông tin hữu ích. Nghiên cứu đã triển khai hai phương pháp: **PCA** (trích xuất đặc trưng, không cần nhãn) và **Chi-Square** (chọn lựa đặc trưng, cần nhãn), với API thống nhất trong lib.reduction.

2. Mục đích của giảm chiều là gì? Giảm bộ nhớ lưu trữ, rút ngắn thời gian huấn luyện và dự đoán, giảm đa cộng tuyến, loại bỏ nhiễu — tất cả trong khi **duy trì độ chính xác phân loại ở mức chấp nhận được**. Đây là sự đánh đổi có chủ đích và có thể kiểm soát được.

3. Giảm chiều có hiệu quả không và đánh đổi gì? Câu trả lời rõ ràng từ thực nghiệm: **rất hiệu quả với sự đánh đổi rất nhỏ**. PCA (n_components = 0.95) đạt được:

Chỉ số	Kết quả
Giảm số chiều	784 → 154 (↓ 80,36%)
Giảm bộ nhớ	358,89 MB → 70,50 MB (↓ 80,36%)
Giảm thời gian fit	19,69 s → 3,60 s (↓ 81,7%)
Giảm thời gian predict	0,0256 s → 0,0052 s (↓ 79,7%)
Suy giảm accuracy	92,57% → 92,29% (↓ 0,28%)

5.2 Ý Nghĩa Thực Tiễn

Kết quả này có ý nghĩa quan trọng cho việc triển khai mô hình học máy trong môi trường thực tế:

Trong môi trường tài nguyên hạn chế: Edge computing, IoT, mobile AI — các nền tảng này thường có RAM và sức mạnh CPU hạn chế. Giảm 80% bộ nhớ và 80% thời gian inference có thể là sự khác biệt giữa có thể triển khai và không thể triển khai.

Trong pipeline sản xuất quy mô lớn: Với hàng triệu mẫu và hàng trăm đặc trưng, việc giảm chiều trước khi huấn luyện có thể cắt giảm chi phí điện toán đám mây đáng kể (tiết kiệm tiền thực).

Trong nghiên cứu và thử nghiệm nhanh: Giảm thời gian fit từ 20s xuống 4s cho phép thử nghiệm nhiều siêu tham số hơn trong cùng khoảng thời gian — tăng tốc chu trình nghiên cứu.

5.3 Hạn Chế và Hướng Phát Triển

Hạn chế của nghiên cứu hiện tại:

- Chỉ đánh giá Logistic Regression; các mô hình phức tạp hơn (SVM, Random Forest, Neural Network) có thể phản ứng khác với giảm chiều.
- Chưa thực nghiệm chi tiết với Chi-Square để so sánh song song đầy đủ với PCA.
- Chưa thử các phương pháp phi tuyến như t-SNE, UMAP, Autoencoder — vốn có thể nắm bắt cấu trúc dữ liệu tốt hơn cho dữ liệu ảnh.
- Thực nghiệm trên MNIST — tập dữ liệu tương đối đơn giản; kết quả có thể khác biệt trên các tập dữ liệu thực tế phức tạp hơn (CIFAR-10, ImageNet, dữ liệu y tế).

Hướng phát triển:

- Mở rộng sang **Kernel PCA** hoặc **Incremental PCA** cho dữ liệu lớn không vừa RAM.
- Thử nghiệm kết hợp giảm chiều với các classifier mạnh hơn như SVM-RBF, XGBoost.
- Nghiên cứu **t-SNE / UMAP** như công cụ phân tích trực quan, kết hợp với PCA làm tiền xử lý (PCA 50 chiều → t-SNE 2 chiều) để giảm chi phí tính toán.

- Mở rộng pipeline sang **MNIST-Fashion**, **EMNIST**, hoặc dữ liệu ảnh y tế để kiểm chứng tính tổng quát của phương pháp.

5.4 Kết Luận Cuối Cùng

Giảm chiều dữ liệu không phải là một bước tùy chọn trong pipeline học máy hiện đại — mà là một **chiến lược kỹ thuật có chủ đích** để cân bằng giữa chất lượng mô hình và hiệu quả tài nguyên. Nghiên cứu này, thông qua thực nghiệm định lượng trên MNIST, đã chứng minh rằng: **không phải mọi chiều dữ liệu đều quan trọng như nhau**, và việc loại bỏ các chiều dư thừa một cách khoa học (dựa trên phương sai trong PCA, hoặc quan hệ với nhãn trong Chi-Square) là hoàn toàn có thể thực hiện mà không hy sinh đáng kể chất lượng dự đoán.

Trong bối cảnh dữ liệu ngày càng nhiều chiều và mô hình ngày càng phức tạp, các kỹ thuật giảm chiều sẽ tiếp tục đóng vai trò thiết yếu — không chỉ như bước tiền xử lý, mà còn như một lăng kính để **hiểu cấu trúc nội tại của dữ liệu**, từ đó xây dựng các mô hình học máy hiệu quả hơn, nhanh hơn, và có khả năng khái quát hóa tốt hơn.